

Project Overview: Simple Uptime Monitoring Service

1. Detailed Description of the Product Problem

Target Audience and Needs

- **Audience:** Freelance web developers, small agency owners, and independent SaaS operators.
- **Needs:** * Automated, 24/7 monitoring of client websites and API endpoints.
 - Instant email notifications when a site becomes unresponsive to minimize downtime.
 - Historical response-time tracking to verify Service Level Agreement (SLA) compliance.

Functional Boundaries

- **Covered:** HTTP HEAD/GET request polling at 60-second intervals, uptime percentage tracking, latency graphing, and email alerts for downtime.
- **Not Covered:** Synthetic user transactions (e.g., simulating logins), global multi-region polling (all checks originate from one server location), or SMS notifications.

2. Key Performance Metrics

Metric	Calculation	Expectation
DAU	Unique user sessions per day	50 (MVP phase)
Active Targets	Total URLs actively monitored	500 URLs
Check Frequency	Pings per URL per day	1,440 (1 per minute)

3. Converting Product Metrics into Technical Parameters

Defining these parameters is critical to provisioning the correct cloud infrastructure and avoiding bottlenecks.

- **RPS (Requests Per Second):** This dictates our internal server load and background worker capacity. Since the system runs on a strict schedule rather than relying on unpredictable user traffic, the math is deterministic.

Internal RPS = $(500 \text{ URLs} \div 60 \text{ seconds}) \approx 8.3 \text{ RPS}$

Peak Expected RPS = $8.3 \text{ RPS} * 10 = 83 \text{ RPS}$

Justification: An 8.3 RPS load is extremely lightweight, meaning we do not need complex distributed worker queues for the MVP.

- **Storage:** We must understand how much time-series data we generate to allocate proper database volume. We need to store the ping result (Status Code, Latency, Timestamp, URL ID) taking roughly 50 bytes per record.

Total Daily Checks = $500 \text{ URLs} \times 1,440 \text{ checks/day} = 720,000 \text{ records/day}$

Daily Storage Added = $720,000 \text{ records} \times 50 \text{ bytes} \approx 36,000,000 \text{ bytes} \approx 36 \text{ MB/day}$

Justification: At 36 MB per day, an annual volume of ~13 GB easily fits onto a standard SSD without requiring specialized analytics databases.

- **Network:** Crucial for calculating bandwidth costs. We will primarily send HTTP HEAD requests.

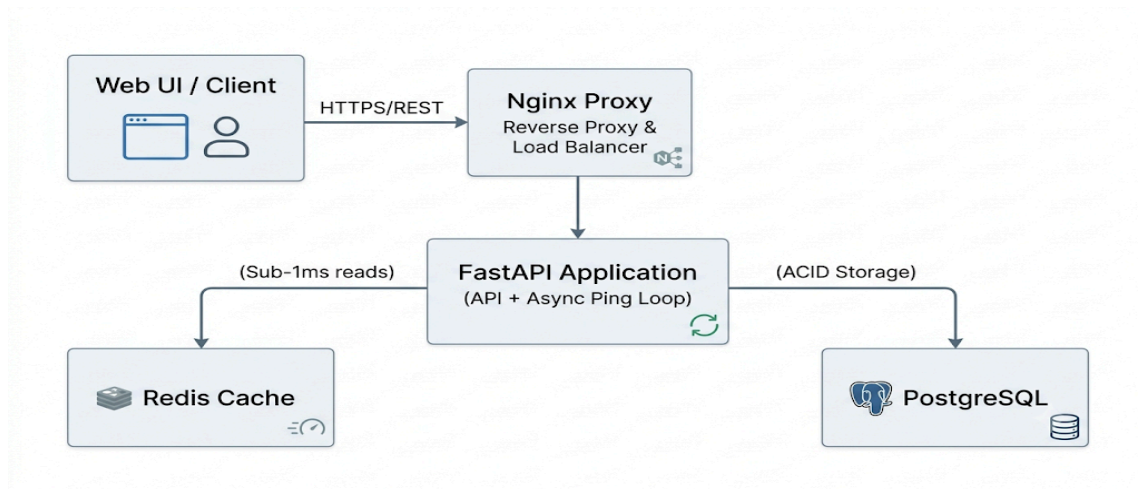
Input Data: $\approx 100 \text{ Bytes per request header}$

Output Data: $\approx 100 \text{ Bytes per response HTTP status}$

Justification: Total network overhead is a few kilobytes per second, operating well below any provider billing limits.

4. Architecture

To keep the system highly cohesive, we will utilize a consolidated single-service architecture that manages both the public API and the background monitoring loops.



Technology Justification

Technology	Reason for Choice
FastAPI (Python)	Native asyncio allows concurrent URL polling within the main application loop without external workers.
PostgreSQL	ACID transactions for user accounts; easily handles time-series logging at our calculated scale.
Redis	In-memory storage for caching the latest URL status, allowing dashboards to load with latency under 1ms.

5. Implementation Constraints and Compromises

Due to strict time boundaries for this MVP submission, several deliberate architectural

compromises were made:

- **No Global Distribution:** Instead of routing checks through multiple geographic regions to verify localized outages, all monitoring runs from a single server location.
- **Consolidated Data Layer:** No specialized analytics database (like ClickHouse) is used. All time-series aggregations are performed directly in PostgreSQL.
- **Single Point of Failure:** There is no database replication or high-availability failover cluster configured.

6. Infrastructure Overview

The application is deployed on a single cloud instance, heavily utilizing containerization for environment consistency.

- **Hosting Selection:** Azure Virtual Machine (B2s Burstable Instance). Utilizes allocated student credits while providing ample resources for 8.3 RPS.
 - **CPU:** 2 Cores
 - **Memory:** 4 GB
 - **Disk:** 30 GB SSD
- **OS Deployment:** Ubuntu 22.04 LTS, running Docker Compose and Nginx.
- **Docker Configuration:** A single docker-compose.yml file orchestrates three services on the same host network: postgres:14, redis:6-alpine, and our custom fastapi-app.
- **Gateway & Security:** Nginx acts as the reverse proxy passing traffic to localhost port 8000, fully secured with TLS 1.3 certificates via Certbot.

Port	Service	Purpose
443	Nginx (HTTPS)	Publicly accessible gateway for web client traffic.
5432	PostgreSQL	Internal database access restricted to localhost container network.
6379	Redis	Internal cache access restricted to localhost container network.